SANIA JOB BOWL · Connecting Talent. Creating the Future.

Employee Identity & Verification Platform

Project Report · November 2026

Summary

A full-stack web application for managing employee records and issuing each employee a unique, revocable QR credential. Anyone who scans the QR sees the **current** status of that employee record (Verified — Active, Inactive, ID expired, Credential expired, Revoked or Invalid). Owners and HR staff manage everything from a restricted dashboard that updates in real time.

Item	Detail
Repository	github.com/saarthaktripathi/saniaemployeeqr (branch `main`)
Stack	Next.js 16 (App Router, React 19, TypeScript strict), Prisma 7, PostgreSQL, Better Auth, Tailwind CSS 4, Zod
Hosting (trial)	Render free web service (Singapore) + Neon free PostgreSQL; auto-deploy on push to `main`
Quality	Lint and typecheck clean; 107/107 unit + integration tests; 18/18 Playwright end-to-end tests; production build passes
Latest additions	Permanent delete of employees (single and bulk), duplicate detection and one-click cleanup, duplicate-safe CSV import

Note: “live monitoring” means live *record status* pushed to HR dashboards. The system does not track employee location, devices or online presence.

1. Features

Area	What it does
HR dashboard	Employee directory with server-side search (name or employee ID), filters (status, department, credential, ID expiry), sorting and pagination; statistic cards and insights; live updates across sessions
Employee records	Add / edit with validation and optimistic concurrency (two people can't silently overwrite each other); immutable, never-reused employee codes such as SJB-2026-001
Lifecycle	Deactivate, reactivate, archive / unarchive (single or bulk, with reasons recorded in the audit log)
Delete (new)	Permanently delete one employee (row trash icon or profile) or many (tick rows → Delete). Requires a reason; all-or-nothing; removes QR credentials so printed QRs show “Invalid”
Duplicates (new)	Import page lists records that look like the same person added twice (e.g. a CSV uploaded twice) and removes all extra copies in one click, keeping one per person. Import can skip rows that already exist
CSV import / export	Import up to 500 rows per file, all rows validated before anything is written; export the filtered directory (formula-injection safe)
QR credentials	Issue, preview, copy, print ID card, reissue and revoke. Each QR holds only an opaque 256-bit token — no personal data
Public verification	`/verify/<token>` shows only company-approved fields and an explicit status; rate-limited and not indexed
Users & roles	Owner, HR Administrator, HR Staff, plus individually grantable permissions; enforced on the server
Security & audit	Argon2id passwords, TOTP MFA (enforceable), DB-backed rate limits, CSP with nonces, CSRF origin checks, append-only audit log
System rules	Company name, ID prefix, ID and credential validity periods, expiry warning window, public fields, retention, MFA policy

2. Roles and permissions

Role	Default access
Owner	Everything, including users, system rules and security audit events
HR Administrator	Read, create, edit, deactivate/reactivate, archive, **delete**, export, issue/revoke QR, read audit log
HR Staff	Read-only by default (directory, QR preview, dashboard). The Owner can grant extra permissions (create, edit, status changes, issue/revoke QR, export, audit read). Delete cannot be granted to HR Staff

Client-side checks only hide buttons; every permission is enforced again inside the server service functions, and denied attempts are written to the audit log.

3. Architecture

A **modular monolith**: one Next.js application containing the UI, HTTP API and domain services, backed by one PostgreSQL database.

```
Browser → proxy (request id, CSP nonce) → Pages (server components) and API route handlers
→ Services (validation, permission check, transaction, audit, outbox) → Prisma → PostgreSQL
→ pg_notify on commit → realtime hub (LISTEN) → /api/events (Server-Sent Events) →
dashboards refresh.
```

Layer	Location	Responsibility
Domain rules	src/features/*	Status transitions, ID validity, verification decision, Zod schemas shared by forms and server
Authorization	src/server/authorization	Role → permission map, grantable extras, deny-by-default can()

Layer	Location	Responsibility
Services	src/server/services	Every read and write: permission check, validation, transaction, audit and outbox rows
HTTP	src/app/api/**	Thin handlers: body limits, CSRF origin check, safe error responses
Auth	src/server/auth	Better Auth (email + password, TOTP), Argon2id, session → actor re-read from DB every request
Realtime	src/server/realtime	LISTEN on PostgreSQL, fan-out over SSE; transactional outbox so rolled-back changes never emit
UI	src/app, src/components	Server components fetch through services; client components call the API

Key flows

- **Create employee:** validate → allocate the next code atomically (INSERT … ON CONFLICT DO UPDATE on a per-prefix, per-year counter) → insert → audit → outbox → commit → dashboards refresh.
- **Issue QR:** revoke any live credential, store SHA-256(token) for lookup and AES-256-GCM(token) so HR can re-print without reissuing. At most one live credential per employee.
- **Verify:** rate limit → format check → hash lookup → decision (revoked > inactive/archived > ID expired > credential expired > verified) → allow-listed fields only.
- **Delete:** check `employee:delete` → confirm every id exists → delete credentials and employees in one transaction → one audit entry per employee (code + reason) → live event.

4. Data model

Table	Purpose
employee	Employee records: UUID id, unique immutable employeeCode, status, ID issue/expiry dates, archivedAt, version
employee_code_counter	Atomic per-prefix, per-year sequence; codes are never reused, even after deletion
employee_credential	QR credentials: token hash, encrypted token (erased on revoke), expiry, revocation
verification_event	Scan analytics with a keyed IP hash; kept (employee link set to null) when an employee is deleted
audit_log	Append-only trail; UPDATE and DELETE blocked by database triggers
outbox_event	Transactional outbox for real-time notifications
company_settings	System rules
user, session, account, twoFactor	Authentication (Better Auth)

5. Removing employees and duplicate cleanup (new)

Where to find it

- **Directory (Dashboard):** trash icon on each row; tick several rows (or the header checkbox) and press **Delete** in the bulk bar.
- **Employee profile: Delete** button next to the status actions; returns to the directory afterwards.
- **Bulk import page** → **Duplicate employees:** groups of matching records with a *Keep* / *Remove* label; **Remove N duplicates** deletes every extra copy at once (up to 1,000 per click); each group also has its own button.
- **Import option “Skip employees that already exist”** (on by default): rows that match an existing record, or an earlier row in the same file, are not created again. The result shows how many were skipped.

How duplicates are recognised

Two records are duplicates when display name, department, job title, business vertical, work location, joining date, phone and work email are all equal (trimmed, case-insensitive). People who share a name but differ in, for example, location are **not** grouped. In each group the record that has a live QR credential is kept (so printed ID cards keep working); otherwise the oldest record is kept.

Safety

- Only Owner and HR Administrator can delete; a reason is mandatory and stored in the audit log with each employee code.
- Bulk deletes are all-or-nothing: if any selected record no longer exists, nothing is deleted.
- Deletion cannot be undone. Use **Archive** when the record may be needed later.

Endpoint	Purpose
POST /api/employees/delete	Body { ids: [...], reason: "..." } — delete one or many (max 1,000)
GET /api/employees/duplicates	List duplicate groups with the suggested record to keep
POST /api/employees/import?skipDuplicates=1	Import CSV, skipping rows that already exist

6. Security

Threat	Mitigation
Broken access control / IDOR	Permission checks inside services; UUID ids; denied attempts audited
Mass assignment	Strict Zod schemas reject unknown keys; role, status and code never accepted from input
SQL injection	Prisma parameterisation; raw SQL only through tagged templates
XSS	React escaping; CSP with per-request nonce, strict-dynamic, frame-ancestors 'none'
CSRF	SameSite HTTP-only cookies; mutations require Origin = APP_BASE_URL
Brute force	Login and MFA 5/min (DB-backed); Argon2id hashing; 12+ character passwords
Session theft	Secure cookies, 8 h expiry, sessions revoked on role/password change or disable
Token guessing / leakage	256-bit tokens; only hash + ciphertext stored; no-referrer and noindex on verify pages
Audit tampering	Triggers block UPDATE/DELETE on audit_log
Denial of service	Body size limits, page size caps, statement timeout, rate limits on bulk actions, import, export and creation

Automated tests are not a substitute for a professional penetration test before handling real employee data at scale.

7. Testing and quality

Check	Result (1 November 2026)
ESLint / TypeScript (strict)	0 errors, 0 warnings / pass
Vitest unit + integration	107 / 107 passed (10 files), including 6 new tests for delete, bulk delete, permissions, duplicate detection and duplicate-safe import
Playwright end-to-end (Chromium, production build)	18 / 18 passed
Production build	pass
Load test, 10,000 employees	Directory p95 115 ms; search p95 ~100 ms; verification p95 52–116 ms; 0% errors; 25 concurrent creations all received unique codes; one change reached 50 live dashboards with p95 17–19 ms

8. Deployment

Current trial setup: **Render** free web service (Singapore) built from `render.yaml`, and **Neon** free PostgreSQL. Build: `npm ci && npm run build`. Start: `npm run start:prod` (applies migrations, creates the first owner once, starts the server). Render redeploys automatically on every push to `main`.

- Free Render services sleep after 15 minutes idle; the first request then takes about a minute.
- Neon free tier: 1 GB storage, 100 compute-hours/month, short restore history — export the directory regularly.
- QR codes embed APP_BASE_URL. Choose the final domain before printing ID cards for real use, then reissue credentials.
- Recommended paid setup later: Render Starter web service + paid Postgres (about USD 13–26/month), custom domain with automatic HTTPS. Requires owner approval.

9. Branding

Place the logo at `public/branding/sania-job-bowl-logo.png` (square PNG, 256×256 px or larger, transparent background). It is used automatically in the header, login page and printed ID cards; until then a neutral placeholder is shown. Replace `src/app/favicon.ico` (or add `src/app/icon.png`) for the browser-tab icon. Commit and push to publish.

10. Local development

```
npm install
cp .env.example .env (generate secrets as described in the file)
npm run db:start (local PostgreSQL on port 5433)
npm run db:deploy
npm run db:seed
npm run dev (http://localhost:3000)
```

Further documentation in the repository: docs/ARCHITECTURE.md, docs/DATA_MODEL.md, docs/SECURITY.md, docs/TESTING.md, docs/DEPLOYMENT.md.